Prvi praktični test iz Algoritama i struktura podataka

Prvi praktični test održan 30.01.2024.

Ime i prezime: _____

Broj indeksa: _____

Broj bodova: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 30 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2023_PopravniFebruar_Test2_GrupaB_IT1g2022**
8. Ne postoji minimum bodova koji se mogu osvojiti iz svakog pojedinačnog zadatka
9. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**
10. **Da bi se test smatrao položenim, neophodno je osvojiti minimum 16 bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Implementacija metode za izgrađivanje najkraće putanje*

Napisati implementaciju metode koja treba da omogući kreiranje trenutno najkraće putanje u okviru Dajkstrinog algoritma

Dodatne napomene za rešavanje zadatka:

- U template-u je zakomentarisana metoda ***private static void BuildShortestPaths(int[] vertices, int[] previousVertices, HashSet<int>[] shortestPaths).***
- U *Main* metodi obezbediti način za testiranje i proveru nove funkcionalnosti.

Zadatak 2 – *Dupliranje neponavljajućih karaktera*

Napisati algoritam vremenske kompleksnosti O(n) koji će iz string-a koji se prosleđuje identifikovati karaktere koji se ponavljaju, duplira sve ostale karaktere ulaznog string-a, te tako izmenjeni string ispisati u konzoli.

Dodatne napomene za rešavanje zadatka:

- Koristiti *Hash* tabelu koja je data u šablonu.
- Implementaciju pisati u okviru zakomentarisane metode ***public static string addMoreDuplicates(string str).***
- Algoritam realizovati tako da **postoji** razlika između malih i velikih karaktera.
- Kako biste string konvertovali u niz karaktera sa kojim se može vršiti manipulacija, koristiti metodu ***ToCharArray()*** po principu: ***str.ToCharArray().***
- Karakteri se ne moraju eksplicitno konvertovati u celobrojni tip prilikom ubacivanja u *Hash* tabelu, ali prilikom kasnijeg ispisivanja koristiti eksplicitno kastovanje ***(char).***
- Redosled karaktera prilikom ispisivanja nije bitan.

Primer:

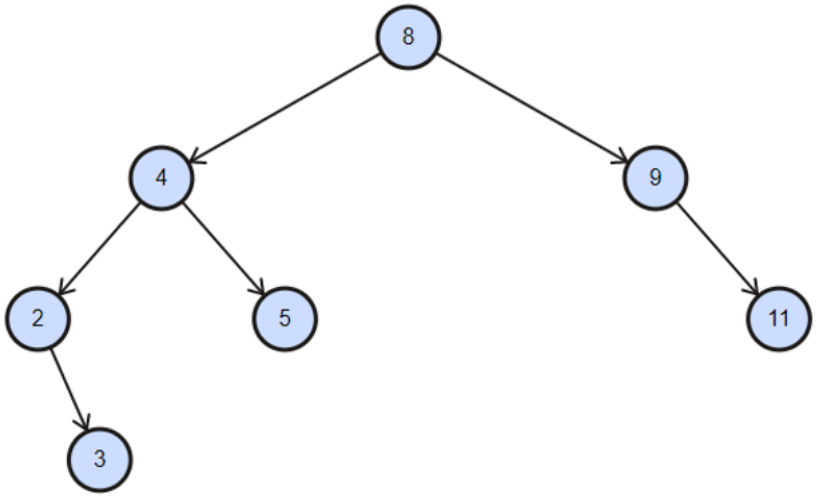
Input 1:	*"Algoritam"*
Output 1:	*"AAllggoorriittaamm"*
	(Nijedan karakter se ne pojavljuje više puta)
Input 2:	*"STRUKTURE podataka"*
Output 2:	*"SSTRUKKTUREE ppooddattakka"*
	(Karakteri „a", „R", „U" i „T" se pojavljuju više puta)

Zadatak 3 – Provera putanje sa sekvencom čvorova

U okviru standardne strukture klase ***BinarySearchTree*** napisati implementaciju metode koja će omogućiti proveru postojanja putanje od korena do lista koja se sastoji iz sekvence čvorova koja je određena nizom.

- Smatra se da putanja sa sekvencom čvorova postoji ukoliko na putanji od korena do lista postoji sekvenca čvorova koji imaju vrednosti koje odgovaraju vrednostima u datom nizu, i nalaze se u redosledu koji je određen poretkom u nizu, pri čemu **ne može biti dodatnih čvorova na putanji**.
- U okviru klase ***BinarySearchTree*** kreirati javnu metodu sa potpisom ***public bool ckeckIfPathExists(int[] arr)***.
- Iz prethodno kreirane javne metode treba da se poziva privatna metoda sa potpisom ***private bool ckeckIfPathExists(Node curr, int[] arr, int index).***
- Privatna metoda treba da sadrži implementaciju algoritma kojim će se omogućiti provera sekvence vrednosti čvorova na putanjama od korena do listova.
- Algoritam **OBAVEZNO** pisati u okviru klase ***BinarySearchTree***

<u>Primeri</u>:



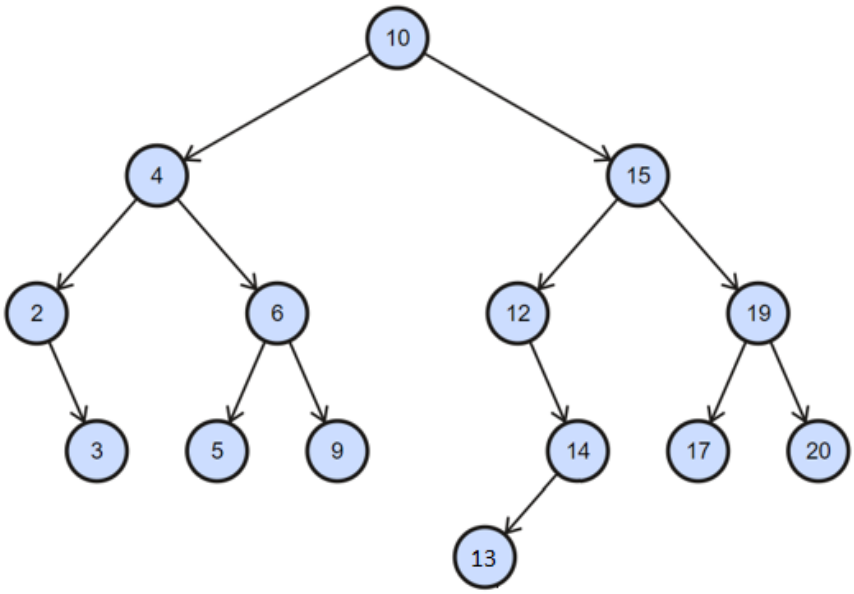
Slika 1 - Primer BST-a

Input 1 za BST sa slike 1: [8, 4, 2, 3]

Output: *true*

Input 2 za BST sa slike 1: [8, 9, 10, 11]

Output: *false*



Slika 2 - Primer BST-a

Input 1 za BST sa slike 2:	[10, 4, 6, 5]
Output:	*true*
Input 2 za BST sa slike 2:	[10, 15, 14, 13]
Output:	*false*